■ Guía de Contribución - Proyecto AXIS

Bienvenido al proyecto AXIS (Asset-Liability eXecution & Investment Simulator). Esta guía te ayudará a configurar tu entorno y comenzar a contribuir de manera segura y efectiva.

■ Tabla de Contenidos

1. [Requisitos Previos](#requisitos-previos)
2. [Configuración Inicial](#configuración-inicial)
3. [Estructura del Proyecto](#estructura-del-proyecto)
4. [Workflow de Desarrollo](#workflow-de-desarrollo)
5. [Estándares de Código](#estándares-de-código)
6. [Comandos Git Útiles](#comandos-git-útiles)
7. [Resolución de Problemas](#resolución-de-problemas)

■ Requisitos Previos

Antes de comenzar, asegúrate de tener instalado:

• **Python 3.8+**: [Descargar Python](https://www.python.org/downloads/)
• **Git**: [Descargar Git](https://git-scm.com/downloads)
• **Kiro (VS Code)**: Editor con extensión Kiro instalada
• **Acceso a Google Drive**: Unidad compartida "Compartida Gestión Financiera (II)"

■ Configuración Inicial

1. Clonar el Repositorio

Abre PowerShell o CMD y ejecuta:

```
# Navegar a tu carpeta de trabajo cd "G:\Unidades compartidas\Compartida Gestión Financiera (II)\Proyecciones" # Clonar el repositorio git clone https://github.com/jorgeejimenez05/AXIS.git # Entrar al directorio cd AXIS
```

2. Configurar Git (Primera vez)

```
# Configurar tu nombre git config --global user.name "Tu Nombre Completo" # Configurar tu email git config --global user.email "tu.email@empresa.com" # Verificar configuración git
```

```
config --list
```

3. Instalar Dependencias de Python

```
# Instalar librerías necesarias pip install -r requirements.txt
```

Esto instalará:
• `pandas`: Manipulación de datos
• `openpyxl`: Lectura/escritura de archivos Excel (.xlsx)
• `xlrd`: Lectura de archivos Excel antiguos (.xls)

4. Verificar la Instalación

```
# Probar el código beta python src/portfolio_loader_beta.py
```

Salida esperada:

```
============================================================ AXIS - Portfolio Loader (Beta)
============================================================ ✓ Portafolio base cargado:
2446 registros ✓ Columnas depuradas: 21 columnas seleccionadas ✓ Portafolio que encaja:
1769 títulos Valor total: $11,331,293,410,529 ✓ Portafolio que no encaja: 83 títulos Valor
total: $385,786,852,728 ✓ Tasas de compra cargadas: 2515 registros ✓ Swaps cargados: 111
contratos ✓ Exportación exitosa: base_proyecciones.xlsx
============================================================ Proceso completado
exitosamente ============================================================
```

■ Estructura del Proyecto

```
AXIS/ ■■■ .kiro/ # Configuración de Kiro ■ ■■■ specs/ # Especificaciones del proyecto ■
■■■ axis-portfolio-inputs/ ■ ■■■ requirements.md # Requerimientos funcionales ■ ■■■
design.md # Diseño técnico ■ ■■■ tasks.md # Plan de implementación ■ ■■■ .config.kiro #
Configuración del spec ■■■ src/ # Código fuente ■ ■■■ portfolio_loader_beta.py # Cargador
de portafolio (versión beta) ■■■ Inputs Py/ # Archivos de entrada (no versionados) ■■■
.git/ # Control de versiones Git ■■■ .gitignore # Archivos ignorados por Git ■■■
README.md # Documentación principal ■■■ CONTRIBUTING.md # Esta guía ■■■ requirements.txt
# Dependencias Python
```

■ Workflow de Desarrollo

Paso 1: Actualizar tu Repositorio Local

Siempre antes de empezar a trabajar:

```
# Asegurarte de estar en la rama main git checkout main # Descargar los últimos cambios git
pull origin main
```

Paso 2: Crear una Rama de Trabajo

Nunca trabajes directamente en `main`. Crea una rama para tu tarea:

```
# Crear y cambiar a una nueva rama git checkout -b feature/nombre-descriptivo # Ejemplos de
nombres de ramas: git checkout -b feature/bloomberg-loader git checkout -b
fix/swap-counterparty-bug git checkout -b docs/update-readme
```

Convención de nombres de ramas:

• `feature/`: Nueva funcionalidad
• `fix/`: Corrección de bugs
• `docs/`: Cambios en documentación
• `refactor/`: Refactorización de código
• `test/`: Agregar o modificar tests

Paso 3: Hacer Cambios

1. Abre el proyecto en Kiro (VS Code)
2. Realiza tus cambios en el código
3. Prueba que todo funcione correctamente

Paso 4: Guardar Cambios (Commit)

```
# Ver qué archivos cambiaron git status # Agregar archivos específicos git add
src/portfolio_loader_beta.py # O agregar todos los cambios git add . # Hacer commit con
mensaje descriptivo git commit -m "feat: Agregar cargador de datos de Bloomberg"
```

Convención de mensajes de commit:

• `feat:` Nueva funcionalidad
• `fix:` Corrección de bug
• `docs:` Cambios en documentación
• `refactor:` Refactorización sin cambiar funcionalidad
• `test:` Agregar o modificar tests
• `chore:` Tareas de mantenimiento

Ejemplos de buenos mensajes:

```
git commit -m "feat: Implementar validación de datos de entrada" git commit -m "fix:
Corregir cálculo de encaje por ramo" git commit -m "docs: Actualizar guía de instalación"
git commit -m "refactor: Simplificar lógica de filtrado de portafolio"
```

Paso 5: Subir Cambios a GitHub

```
# Subir tu rama al repositorio remoto git push origin feature/nombre-de-tu-rama
```

Paso 6: Crear Pull Request

1. Ve a https://github.com/jorgeejimenez05/AXIS
2. Verás un botón "Compare & pull request"
3. Completa la descripción del Pull Request:
- ¿Qué cambios hiciste?
- ¿Por qué son necesarios?
- ¿Cómo probaste los cambios?
4. Solicita revisión de un compañero
5. Espera aprobación antes de hacer merge

Paso 7: Después del Merge

```
# Volver a la rama main git checkout main # Actualizar con los cambios mergeados git pull
origin main # Eliminar tu rama local (opcional) git branch -d feature/nombre-de-tu-rama
```

■ Estándares de Código

Python

• **PEP 8**: Seguir el estilo de código estándar de Python
• **Docstrings**: Documentar todas las funciones y clases
• **Type hints**: Usar anotaciones de tipo cuando sea posible
• **Nombres descriptivos**: Variables y funciones con nombres claros

Ejemplo:

```
def load_portfolio_base(self, filename: str = "base portafolio python.xlsx") ->
pd.DataFrame: """ Carga el archivo base del portafolio desde Excel. Args: filename: Nombre
del archivo Excel con el portafolio Returns: DataFrame con los datos del portafolio Raises:
FileNotFoundError: Si el archivo no existe """ ruta = f"{self.base_path}/{filename}"
self.df = pd.read_excel(ruta, header=1) print(f"✓ Portafolio base cargado: {len(self.df)}
registros") return self.df
```

Comentarios

• Comentarios en español (idioma del equipo)
• Explicar el "por qué", no el "qué"
• Mantener comentarios actualizados

Manejo de Errores

```
try: df = pd.read_excel(ruta, header=1) except FileNotFoundError: print(f"■ Error: No se
encontró el archivo {ruta}") raise except Exception as e: print(f"■ Error inesperado: {e}")
raise
```

■■ Comandos Git Útiles

Información del Repositorio

```
# Ver estado actual git status # Ver historial de commits git log --oneline # Ver historial con gráfico git log --oneline --graph --all # Ver ramas locales git branch # Ver ramas remotas git branch -r # Ver todas las ramas git branch -a
```

Trabajar con Ramas

```
# Crear nueva rama git branch nombre-rama # Cambiar a una rama git checkout nombre-rama # Crear y cambiar a nueva rama (atajo) git checkout -b nombre-rama # Eliminar rama local git branch -d nombre-rama # Eliminar rama remota git push origin --delete nombre-rama
```

Ver Cambios

```
# Ver diferencias no guardadas git diff # Ver diferencias de un archivo específico git diff src/portfolio_loader_beta.py # Ver diferencias entre ramas git diff main feature/mi-rama
```

Deshacer Cambios

```
# Descartar cambios en un archivo (antes de add) git restore src/portfolio_loader_beta.py # Descartar todos los cambios no guardados git restore . # Quitar archivo del staging (después de add) git restore --staged src/portfolio_loader_beta.py # Deshacer último commit (mantener cambios) git reset --soft HEAD~1 # Deshacer último commit (descartar cambios) ■■ PELIGROSO git reset --hard HEAD~1
```

Sincronizar con GitHub

```
# Descargar cambios sin aplicarlos git fetch origin # Descargar y aplicar cambios git pull origin main # Subir cambios git push origin nombre-rama # Forzar push (■■ usar con precaución) git push --force origin nombre-rama
```

■ Resolución de Problemas

Problema: No tengo acceso a la unidad G:

Solución:

1. Verificar que tienes acceso a Google Drive
2. Sincronizar la unidad compartida "Compartida Gestión Financiera (II)"
3. Verificar que la ruta existe: `G:\Unidades compartidas\Compartida Gestión Financiera (II)\Proyecciones\Proyecto modelo proyecciones`

Problema: `pip` no está instalado

Solución:

```
# Verificar instalación de Python python --version # Reinstalar pip python -m ensurepip --upgrade
```

Problema: `git` no está instalado

Solución:
1. Descargar Git desde https://git-scm.com/downloads
2. Instalar con opciones por defecto
3. Reiniciar PowerShell/CMD
4. Verificar: `git --version`

Problema: No puedo hacer push a GitHub

Solución:

```
# Verificar configuración git config --list # Configurar credenciales git config --global user.name "Tu Nombre" git config --global user.email "tu@email.com" # Si pide autenticación, usar Personal Access Token # Ir a GitHub → Settings → Developer settings → Personal access tokens
```

Problema: Conflictos al hacer `git pull`

Solución:
1. Git marcará los archivos con conflictos
2. Abrir los archivos y buscar:
`
<<<<<<< HEAD
Tu código
=======
Código del repositorio
>>>>>>> commit-hash
`
3. Decidir qué código mantener
4. Eliminar las marcas `<<<<<<<`, `=======`, `>>>>>>>`
5. Guardar el archivo
6. Hacer commit:
`bash
git add archivo-con-conflicto.py
git commit -m "fix: Resolver conflicto en archivo-con-conflicto.py"

`

Problema: Módulo no encontrado al ejecutar Python

Solución:

```
# Verificar que estás en el directorio correcto pwd # Reinstalar dependencias pip install
-r requirements.txt # Verificar instalación pip list
```

Problema: Error al leer archivos Excel

Solución:

```
# Instalar/actualizar openpyxl pip install --upgrade openpyxl # Para archivos .xls antiguos
pip install --upgrade xlrd
```

■ Contacto y Soporte

Si tienes problemas que no puedes resolver:

1. **Revisar esta guía** completa
2. **Buscar en el historial de issues** en GitHub
3. **Crear un nuevo issue** en GitHub con:
- Descripción del problema
- Pasos para reproducirlo
- Mensajes de error (capturas de pantalla)
- Tu sistema operativo y versión de Python
4. **Contactar al equipo** de Gerencia Financiera

■ Trabajar con Especificaciones en Kiro

Ver las Tareas Pendientes

1. Abrir `.kiro/specs/axis-portfolio-inputs/tasks.md`
2. Ver el estado de cada tarea:
- `[ ]` = No iniciada
- `[~]` = En cola
- `[-]` = En progreso
- `[x]` = Completada

Ejecutar una Tarea con Kiro

En el chat de Kiro:

```
Ejecuta la tarea 1.1 del spec axis-portfolio-inputs
```

Kiro:
• Leerá los requerimientos y diseño
• Implementará la tarea
• Actualizará el estado en tasks.md
• Hará commit de los cambios

Actualizar el Spec

Si necesitas modificar requerimientos o diseño:

1. Editar `.kiro/specs/axis-portfolio-inputs/requirements.md` o `design.md`
2. Hacer commit de los cambios
3. Informar al equipo de los cambios

■ Checklist Antes de Hacer Push

• [ ] El código funciona correctamente
• [ ] Probé el código con datos reales
• [ ] Agregué/actualicé docstrings
• [ ] El código sigue los estándares de PEP 8
• [ ] Hice commit con mensaje descriptivo
• [ ] Actualicé documentación si es necesario
• [ ] No incluí datos sensibles o credenciales
• [ ] No incluí archivos grandes innecesarios

■ Recursos Adicionales

• **Git**: https://git-scm.com/doc
• **Python PEP 8**: https://pep8.org/
• **Pandas**: https://pandas.pydata.org/docs/
• **GitHub Flow**: https://guides.github.com/introduction/flow/

■ ¡Gracias por Contribuir!

Tu trabajo es fundamental para el éxito del proyecto AXIS. Si tienes sugerencias para mejorar esta guía, no dudes en proponer cambios.

Equipo AXIS - Gerencia Financiera